# ModbusControl - User manual

For version 2.0.23

## Table of contents


1 Install ... 2
2 Launching ModbusControl ... 2
3 The .mbc Project File ... 2
4 Using ModbusControl ... 2
4.1 Load, Edit and Save .mbc Project Files ... 2
4.2 Import and Export CSV files ... 3
4.3 Connect, Disconnect, Start Polling, Stop Polling ... 3
4.4 Find / Filter ... 4
4.5 Project Configuration ... 4
4.6 Options ... 6
4.7 Custom Modbus Queries ... 8
4.8 Diagnostics counters ... 9
4.9 Logging ... 9
4.10 The Help menu ... 9
4.11 The tab pages ... 9
4.12 Bit fields ... 10
4.13 Write a register ... 10
4.14 The status bar ... 11
5 Mbc project file validation with XSD schema ... 11
6 Logging ... 12
7 Modbus/TCP bridge ... 12
8 The Modbus protocol, function codes and word order ... 12
9 ModbusControl command line parameters ... 13
10 ModbusExport tool ... 14
11 ModbusImport tool ... 16
12 The Import/Export format ... 18
13 More about Modbus ... 18
13.1 Editing the XML in a .mbc project file ... 19
13.2 The <modbuscontrol> root element ... 19
13.3 The <size> element ... 19
13.4 The <modbus> element ... 20
13.5 The <tab> element ... 21
13.6 The <include> element ... 22
13.7 The <var> element ... 23
13.8 The <text> element ... 24
13.9 The <longdescr> element ... 25
13.10 The <type> element ... 25
13.11 The <enum> element ... 26
13.12 The <field> element ... 26
13.13 The <fcol> and <bcol> elements ... 27
13.14 The <autostart> element ... 28
13.15 The <modbustcpserver> element ... 28
13.16 XML comments ... 28

# 1 Install

The installer places all necessary program files in the default installation directory: *\Program Files\ModbusControl*. It also installs an example project file ”**Demo.mbc**”, a PDF user manual and a PDF revision history document.

In addition, the installer registers a Windows file association for the [**.mbc]** extension. As a result, double-clicking any .mbc file will automatically start ModbusControl and load the selected project.

**Note:** ModbusControl requires the Microsoft .NET Framework version 4.7.2 or later.

# 2 Launching ModbusControl

The most common way to start ModbusControl is by double-clicking an .mbc project file. This automatically:

- Launches the application
- Loads the selected project
- Opens the connection
- Begins communication

Alternatively, you can start ModbusControl via the Windows Start Menu. Once the application is open, you can:

- Use **File > Open** or select a file from **Recent Project Files**
- Click **Open Connection**
- Click **Start Polling**

For advanced usage, ModbusControl can also be launched with command-line parameters to override the connection settings defined in the .mbc file. Refer to the *[ModbusControl command line parameters]* documentation for details.

# 3 The .mbc Project File

An .mbc project file is an XML-based configuration file that defines:

- Connection settings such as TCP or serial port parameters, slave ID, timeouts, and sample rate
- Tab pages containing Modbus register definitions
- Modbus/TCP bridge server configuration
- Preferred window size

# 4 Using ModbusControl

## 4.1 Load, Edit and Save .mbc Project Files

The **File** menu provides commands to open, reload and edit .mbc project files.
You can also drag and drop an .mbc file onto the ModbusControl tab-pages area to load it.

To edit the current project file, use **Edit project file**. ModbusControl will launch the text editor associated with .txt files in Windows. Editing Modbus registers directly inside ModbusControl is

not supported.

The **Save project file…** command allows you to update the project file with the current connection and protocol settings, the current Modbus/TCP bridge settings, and the current window size. The dialog that follows lets you choose which items to save.

The **Save project file…** dialog also includes an **Update XML to current version** option. This updates the .mbc file to the latest XML format. By converting all tags to lowercase, removing or replacing obsolete elements, and reformatting the file.

If ModbusControl was started with command-line overrides, the **Save project file…** dialog will display a warning:

> ***"Some connection parameters are overridden by the command line! If you save the current configuration to the mbc project file you may overwrite some parameters with the command line values!"***

## 4.2 Import and Export CSV files

### Export registers to file

The export function reads the values of the registers from the Modbus device and saves them to a CSV text file.

You can choose whether to export only the registers of the current tab page or all tab pages. Select the desired value format (int, uint, hex, …) and specify the output file. A **Continue on error** option is also available.

Click **Start** to begin the export. ModbusControl will read the actual register values from the device and write them to the CSV file. When the export completes, a summary line shows how many registers were exported and how many errors occurred. Registers defined as *write-only* are not exported because they cannot be read from the Modbus device.

The order in which tab pages are exported can be controlled using the `<exportorder>` element in the .mbc file. This is useful when you want registers to appear in a specific sequence in the CSV file - for example, placing setup registers first to ensure correct import behavior.

See The <tab> element - `<exportorder>` for more information.

See The Import/Export format for details on the CSV file structure.

### Import registers from file

The import function reads register numbers and values from a CSV file and writes those values to the Modbus device.

Click **Select file** to choose the input CSV file. A **Continue on error** option is also available.

Click **Start** to begin the import. Registers marked as *read-only* in the CSV file are not imported. When the import completes, a summary line shows how many registers were successfully written, how many were non-writeable, and how many errors occurred.

See The Import/Export format for more information on the CSV file structure.

## 4.3 Connect, Disconnect, Start Polling, Stop Polling

Use the **Connection** menu (or the toolbar buttons) to connect to a Modbus device. When you connect, ModbusControl opens the serial port or establishes the TCP connection, depending on the project settings.

Use the **Start polling**, **Stop polling**, and **Poll once** commands to control how ModbusControl reads the Modbus registers.

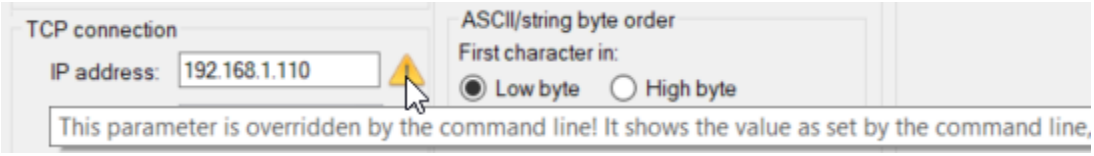
## 4.4 Find / Filter

The **View** menu contains a command to show the **Find/Filter** ribbon. This feature allows you to filter registers based on their register number or on text in the register description.

## 4.5 Project Configuration

The **Project configuration** command in the **Extra** menu opens the Project Configuration window. Here you can configure project parameters such as connection type, baud rate, IP address, sample rate, endian settings, and more.

If a parameter is overridden by a command-line option, a warning icon is displayed next to that parameter. Hovering the mouse over the icon shows a tooltip with additional information.




| | | |
|---|---|---|
| Connection: | Serial/RTU: | A serial-port connection using the Modbus RTU protocol. |
| | Modbus/TCP: | A TCP network connection using the Modbus/TCP protocol. |
| | RTU over TCP: | A TCP network connection that transports the Modbus RTU protocol over TCP. This is useful when working with serial-to-network bridge devices. |
| Serial interface: | Port: | The serial port name, such as **COM1**. |
| | Baudrate: | The communication speed in bits per second. Common values include **9600** and **19200**. |
| | Parity: | The parity setting for the serial connection. |
| | EOM timeout: | End-of-message timeout. Normally, ModbusControl uses the *implied length* mechanism to detect the end of response frames. However, when using a Modbus/TCP bridge and unknown function codes, this timeout is required to detect the end of the response frame. Default: **100 ms**. |
| | One-wire echo suppression: | Suppresses the command being echoed back to ModbusControl. This is only needed if the serial hardware causes ModbusControl to receive an echo of the command it sent. |
| TCP connection: | IP address: | The IPv4 address of the device, such as 192.168.1.52.<br>Use 127.0.0.1 for local loopback when the computer needs to communicate with itself. |
| | Port: | The TCP port number. Modbus/TCP normally uses the registered port **502**. |

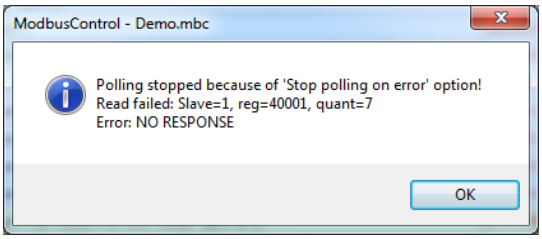
| | | |
|---|---|---|
| Slave ID: | A value from **0 to 255**. Also referred to as the **slave address** or **unit identifier**. | |
| Response timeout: | The maximum time ModbusControl waits for the first bytes of the response to arrive, in milliseconds. | |
| Maximum frame size: | ModbusControl constructs its queries so that neither the request nor the response will exceed this number of bytes. This allows communication with serial devices that have small receive buffers. Valid range: **13 to 256 bytes**. | |
| Inter-frame delay: | The delay between receiving the last byte of a response and sending the next query. Modbus slaves on a serial multidrop network require this delay, as defined in the Modbus specification.<br>**On serial port:** A value of **0 (auto)** applies the default delay of four character times (for example, about 4 ms at 9600 baud). Any value greater than 0 specifies the delay in milliseconds.<br>**On TCP connection:** An optional delay in milliseconds.<br>This delay can also be used to reduce the load on the Modbus device. | |
| Function code to write holding registers: | | |
| | Use FC6 only: | Always use function code **6** (*Write Single Register*). |
| | Use FC16 only: | Always use function code **16** (*Write Multiple Registers*). |
| | Auto select FC6 or FC16: | Automatically uses **FC6** when writing a single register and **FC16** when writing multiple registers at once. |
| 32-bit integer word order: Defines how a 32-bit integer value is stored across two 16-bit registers: | | |
| | Low-High: | The lower 16 bits are stored in the first register, and the higher 16 bits in the second register.<br>This is the most commonly used format. |
| | High-Low: | The higher 16 bits are stored in the first register, and the lower 16 bits in the second register. |
| 32-bit float word order: Defines how a 32-bit IEEE-754 floating-point value is stored across two 16-bit registers: | | |
| | Low-High: | The lower 16 bits are stored in the first register, and the higher 16 bits in the second register.<br>This is the most commonly used format. |
| | High-Low: | The higher 16 bits are stored in the first register, and the lower 16 bits in the second register. |
| String/ASCII byte order: Determines how characters are arranged within a 16-bit register: | | |
| | First char in low byte: | A register with the value **0x4142** will be shown as **“BA”**. |

| | | |
|---|---|---|
| | First char in high byte: | A register with the value **0x4142** will be shown as **"AB"**. This is the most commonly used mode. |
| String encoding/decoding: | | Selects the character encoding used for reading and writing strings. The dropdown lists all supported encodings. Common choices include **US-ASCII**, **UTF-8**, and **Windows-1252**. |
| Sample rate (ms): | The registers on the current tab page are read from the Modbus device at a fixed interval. The sample rate defines this interval in **milliseconds**. | |
| Holding registers start: | Shows the numbering scheme used for the Holding Registers, as configured in the .mbc project file.<br>40001:<br>The most commonly used scheme. Ranges **40001–49999** and **400001–465536** are supported and can be used simultaneously. Both **40001** and **400001** refer to the first holding register, which is addressed as **0000** in the protocol.<br>40000:<br>Ranges **40000–49999** and **400000–465535** are supported and can be used simultaneously. Both **40000** and **400000** refer to the first holding register, addressed as **0000** in the protocol.<br>400001:<br>Six-digit numbering scheme. Range **400001–465536** is supported. **400001** refers to the first holding register, addressed as **0000** in the protocol.<br>400000:<br>Six-digit numbering scheme. Range **400000–465535** is supported. **400000** refers to the first holding register, addressed as **0000** in the protocol. | |
| Modbus/TCP bridge server: | Configures the built-in Modbus/TCP-to-Modbus-device bridge. See Modbus/TCP bridge for more details. | |
| | Enable: | Enables or disables the bridge. |
| | TCP port: | The TCP port the bridge listens on. Default: **502**. |

## 4.6 Options

Opens the **Options** window, which contains several general program settings:

| | |
|---|---|
| Beep on input error: | Enables or disables an audible beep when the user enters an invalid value. |
| Move to next row on Enter: | **On:** The cursor moves to the next row after entering a new value or pressing **Enter** in the grid.<br>**Off:** The cursor remains on the current row.<br>Use **Ctrl** + **Enter** to temporarily perform the opposite action. |

| | |
|---|---|
| Auto-start edit mode on typing: | **On:** Typing any character immediately switches the grid into edit mode.<br>**Off:** The user must double-click the value field or press **F2** to enter edit mode. |
| Show register properties column: | **On:** Displays an additional **Properties** column showing the register type (e.g., *int16*), read/write mode, number of decimals, and other attributes. |
| Multiline rows with auto row size: | **On:** Allows long descriptions or text containing line breaks (CR/LF) to wrap onto multiple lines.<br>**Off:** All description text is shown on a single line and truncated if necessary. |
| Clear Find/filter on tab change: | **On:** The active filter is cleared when switching to another tab page.<br>**Off:** The filter remains active across tab pages. |
| Stop polling on error: | Enables or disables automatic stopping of polling when a read error occurs. If polling is stopped due to an error, a message similar to the following is shown:<br>  |
| Ignore form size on project reload: | **On:** ModbusControl ignores the form size stored in the .mbc project file when the project is reloaded. The window keeps its current size.<br>**Off:** The form size from the project file is applied.<br>Default: **Off**. |
| Use XSD validation schema on project load: | **On:** The .mbc project file is validated against the XSD schema when loaded. See Mbc project file validation with XSD schema for more details.<br>**Off:** The project file is loaded without schema validation.<br>Default: **On**. |
| Use custom editor: | Allows you to configure an external text editor for editing .mbc project files. By default, ModbusControl uses the editor associated with .txt files in Windows.<br>• Supports one parameter: **%file**, which is replaced with the full path of the file to edit.<br>• Enclose both the editor path and %file in **double quotes**.<br>Example (using Windows Notepad):<br>`"C:\windows\system32\notepad.exe" "%file"` |

| | |
|---|---|
| Logger capacity: | Configures the maximum number of log lines kept in memory. When the limit is reached, the oldest entries are automatically removed. |
| Check for program updates: | Enables or disables the automatic check for updates each time the program starts. |

## 4.7 Custom Modbus Queries

The **Custom Modbus Queries** form provides a flexible grid interface for defining a wide range of Modbus read and write operations. It is especially useful for testing and diagnostics, such as reading up to **125 registers** in a single operation, validating device behavior, or troubleshooting communication issues.

You can perform the following operations:

- **Read Input Registers** (Function Code **0x04**)
- **Read Holding Registers** (Function Code **0x03**)
- **Write a Single Holding Register** (Function Code **0x06**)
- **Write Multiple Holding Registers** (Function Code **0x10**)
- **Read Discrete Inputs** (Function Code **0x02**)
- **Read Coils** (Function Code **0x01**)
- **Write a Single Coil** (Function Code **0x05**)
- **Write Multiple Coils** (Function Code **0x0F**)
- **Send a Custom Query**, where you manually construct and send a Modbus request tailored to your specific testing needs.

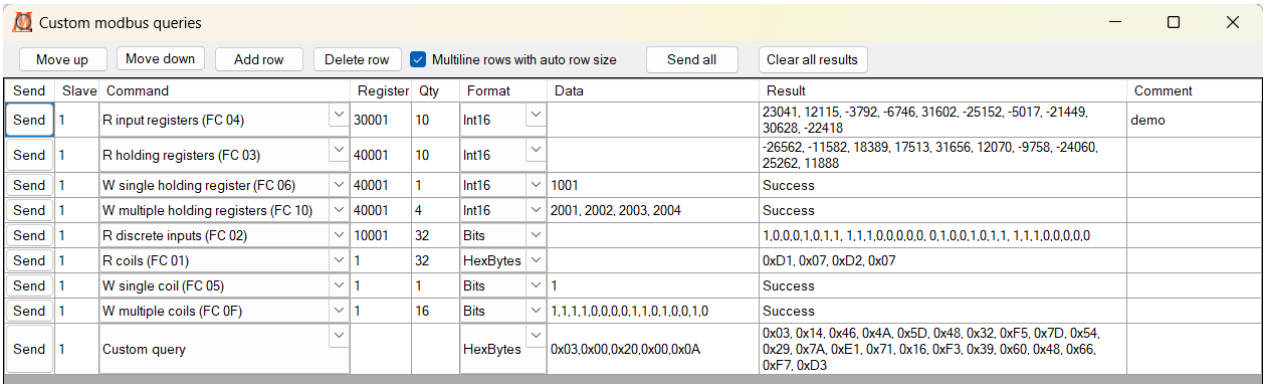
Data can be displayed in multiple formats, including signed and unsigned 16-bit and 32-bit integers, hexadecimal values, bit fields, or raw hex bytes.

The **Custom Query** operation allows you to manually construct and send a Modbus request. The query is defined as a comma-separated list of bytes, entered in either:

- **Decimal format:** 1,3,0,0,0,10
- **Hexadecimal format:** 0x01,0x03,0x00,0x00,0x00,0x0A

This feature is ideal for:

- Testing non-standard or vendor-specific commands
- Simulating edge cases
- Verifying device behavior under specific conditions
- Sending raw frames for advanced troubleshooting




The grid includes buttons to move, insert, and delete rows.

Register numbering in this tool is **independent** of the numbering scheme used in the .mbc project file. It follows the standard Modbus conventions:

- **0xxxx (Coils):** 1-bit data, read/write access.
- **1xxxx (Discrete Inputs):** 1-bit data, read-only access.
- **3xxxx (Input Registers):** 16-bit data, read-only access.
- **4xxxx (Holding Registers):** 16-bit data, read/write access.

## 4.8 Diagnostics counters

The **View** menu includes a **Diagnostics counters** command that opens a window displaying a set of counters. These counters show the number of reads, writes, errors, and various other communication statistics, providing a quick overview of Modbus activity and performance.

## 4.9 Logging

The **View** menu also includes a **Logger** command, which opens the logger window. The logger can display all reads, writes, errors, warnings, and additional diagnostic information. See Logging for more details.

## 4.10 The Help menu

The **Help** menu provides quick access to key resources and information within ModbusControl:

- **User Manual** - Opens the ModbusControl user manual.
- **Demo Project -** Loads a sample project file named **Demo.mbc**, useful for exploring the tool's capabilities and learning how to structure your own projects.
- **Check for Updates** - Initiates a check for software updates. This feature is only available within the **E.D.&A internal network** and will not function outside that environment.
- **About Window** - Displays the current program version along with additional application details.

## 4.11 The tab pages

The tab pages display the Modbus registers, their description text, and the most recently read value — or an error message if the read operation failed.

Right-clicking any cell in the *Register*, *Description*, or *Properties* columns opens a context menu that allows you to change the background **color** of the selected row. This is useful for visually highlighting registers you want to track or prioritize.
The same menu also includes a **Copy line** option, which copies the entire row to the clipboard. Alternatively, you can press **Ctrl + C** after selecting a cell in any of these columns to copy the full row as well.

Right-clicking a cell in the *Value* column opens a menu that allows you to change how the register value is displayed. The available display formats are:

- **Int:** Shows the value as a signed integer
- **UInt:** Shows the value as an unsigned integer
- **Hex:** Shows the value in hexadecimal format (e.g., 0x1234)
- **Binary:** Shows the value in binary format (e.g., 0b10010110).
- **Float:** Interprets the value as a 32-bit IEEE 754 floating-point number (applicable only to 32-bit registers).
- **ASCII:**
  16-bit register → 2 ASCII characters.
  32-bit register → 4 ASCII characters.
  Non-printable characters are shown using escape sequences. For example, a register with value `0x0141` is displayed as **`"\x01A"`.**
- **String:** Displays the value as text.

A check mark indicates the currently selected format.
Use the **Copy value** menu item to copy only the register's value to the clipboard.
You can also press **Ctrl + C** after selecting a cell in the *Value* column to perform the same action.

### 4.12 Bit fields

Some registers may be defined with associated **bit fields**. Registers that contain bit fields display a **[+]** icon in front of them.

- Click the **[+]** icon to expand the register and view its bit fields.
- Click the **[–]** icon to collapse the bit fields again.

### 4.13 Write a register

If a register or bit field is writeable, its value can be edited by double-clicking the value field or by pressing **F2** on the selected line. If the **Auto-start edit mode on typing** option is enabled, editing begins automatically when the user types a numeric or character key.

You can enter a new value in any of the following formats:

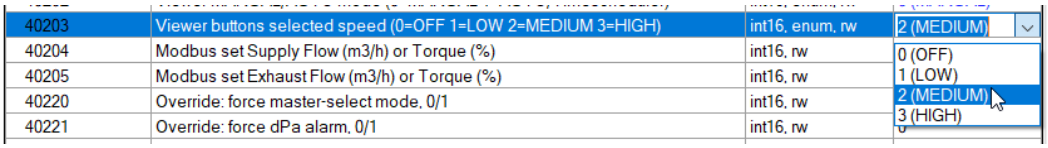
- **Decimal:** positive or negative (e.g., 123, -45)
- **Hexadecimal:** starting with 0x (e.g., 0x1234)
- **Binary:** starting with 0b (e.g., 0b1101)
- **ASCII:** starting with a double quote (e.g., "AB")
- **String:** starting with a double quote (e.g., "This is a string text")

The entered value is validated to ensure it is allowed and fits within the width of the register or bit field. Press **Escape** to cancel the input.

When you change the value of a **bit field**, ModbusControl automatically recalculates the full

register value based on the current values of all bit fields within that register.

If the value is displayed as an **enum**, a dropdown list of predefined values appears during editing. You may still enter any other valid value manually.




When entering a new value, you may encounter **error messages** such as <CONNECTION NOT OPEN>, <REGISTER IS READ-ONLY> or <INVALID VALUE>.

If the **Move to next row on Enter** option is enabled, the selection automatically moves to the next row after the new value is entered.

### 4.14 The status bar

During normal operation, the status bar displays:

- The number of successful reads
- The number of times ModbusControl did not receive a response
- The number of times an invalid response was received
- Various informational messages, shown for a few seconds

The status bar changes color to indicate system state:

- **Blue:** No connection or no polling is active
- **Red:** An error message is being displayed

Hover the mouse cursor over the status bar to view the most recent informational or error message again.

## 5 Mbc project file validation with XSD schema

ModbusControl can use an XML validator to verify the structure of .mbc project files. When a project file is loaded, it is first validated against an **XSD schema**. If validation fails, an error message is shown, but you may choose to ignore the error and load the file anyway.

The schema validation enforces the following rules:

- Only **lower-case XML tags** are allowed
- Only **currently supported XML elements** are permitted
- Obsolete or deprecated elements have been removed or replaced

Previously, many .mbc project files contained unnoticed errors because ModbusControl simply ignored unknown XML elements. With schema validation enabled, this is no longer allowed — only valid, supported elements may appear in the file.

If necessary, schema validation can be disabled in the **Options** menu.

ModbusControl can convert older .mbc project files to the new format. See Load, Edit and Save .mbc Project Files for more information.

Please ensure that all your .mbc project files are updated to conform to the validation rules over time. When sending a project file to an external partner, verify that it passes schema validation.

# 6 Logging

ModbusControl includes a built-in logging system that can record a wide range of events and communication details. The logger can capture:

- **Errors** — such as response timeouts
- **Warnings**
- **Information messages** — for example, when a connection is opened
- **Trace TX/RX** — raw Modbus messages sent to and received from the device
- **Read queries** — Modbus read commands used to fetch register values
- **Write queries** — user-initiated changes to register values
- **Bridge info** — clients connecting to or disconnecting from the Modbus/TCP bridge
- **Bridge data** — Modbus/TCP frames received and sent through the bridge
- **Bridge decode** — Modbus/TCP frames decoded into human-readable messages, such as *"Slave=1, Read holding registers: addr=100, quant=15"*

You can start or stop the logger at any time, clear the log, copy the log contents to the Windows clipboard (in **CSV** format), or save the log to a file — also in **CSV** format.

# 7 Modbus/TCP bridge

ModbusControl includes a built-in **Modbus/TCP server** that acts as a bridge to the connected Modbus device. This allows Modbus/TCP clients — such as **ModbusScope** — to communicate with the device **simultaneously** with ModbusControl. Multiple Modbus/TCP clients, or even multiple instances of ModbusControl, can access the same device at the same time. The bridge works with **all connection types**, not only serial lines.

A query received from a Modbus/TCP client is buffered until the communication line to the Modbus device becomes available. Once the line is free, the query is forwarded. As soon as a response is received, it is returned to the Modbus/TCP client that issued the query. If a response timeout occurs, ModbusControl sends the appropriate Modbus exception message back to the client.

The Modbus/TCP protocol includes a **Unit Identifier** byte. This value is used as the **slave ID** when forwarding the query to the Modbus device. To receive a response, the Unit Identifier must match the slave ID of the connected device. Some Modbus/TCP clients ignore this field and send values such as `0x00` or `0xFF`.

Because the serial line to the Modbus device has limited bandwidth, the number of Modbus/TCP clients and their sampling rates is inherently limited.

The Modbus/TCP bridge can be configured through the **Options → Project Configuration** menu.

# 8 The Modbus protocol, function codes and word order

ModbusControl supports the **Modbus RTU (binary)** and **Modbus/TCP** protocols. **ASCII mode is not supported.**

ModbusControl currently supports reading and writing **Holding Registers** using the following function codes:

- **03** — Read Holding Registers

- **06** — Write Single Register
- **16** — Write Multiple Registers

**32-bit values**

A 32-bit value consists of **two consecutive 16-bit registers**. These can be transferred in either:

- **Low-High word order**
- **High-Low word order**

E.D.&A recommends **Low-High** word order. This is configured in the project settings via **"32-bit integer word order"** or in the .mbc project file using the `<int32endian>` element.

**Floating-point values**

ModbusControl supports **32-bit IEEE-754 single-precision floats**, which also require two consecutive 16-bit registers. These can also be transferred in **Low-High** or **High-Low** word order.

E.D.&A again prefers **Low-High** word order. This is configured via **"32-bit float word order"** or in the .mbc file using the `<float32endian>` element.

**Register addressing**

In Modbus protocol messages, register addresses start at **zero**. The first Holding Register is addressed as **0** in the query.

For historical reasons, Holding Registers are often documented as **40001–49999**. The base number used for display is defined in the .mbc project file via the `<holdingregisterstart>` element. Common values include **40001**, **400001**, **40000**, or **400000**, typically chosen to match the device documentation.

**Optimized reading**

ModbusControl attempts to read as many consecutive registers as possible using a single **Read Holding Registers** query to improve performance.

The maximum number of registers per query is limited by the **Maximum frame size** parameter. ModbusControl ensures that queries and responses never exceed this size.

If a device returns an **Illegal Data Address** exception, it may indicate that the query requested more registers than the device supports. To display available registers and identify missing ones, ModbusControl automatically re-reads them **one by one**.

**Writing registers**

To write Holding Registers, ModbusControl uses:

- **06** — Write Single Register
- **16** — Write Multiple Registers

See Function code to write holding registers in Project Configuration for more details.

# 9 ModbusControl command line parameters

Command-line options allow you to override connection parameters when starting ModbusControl.

Example usage:

*Modbuscontrol Test1.mbc -connection serialrtu -port COM6 -baudrate 19200*

*Modbuscontrol Test1.mbc -connection modbustcp -ipaddress 192.168.1.123*

The .mbc filename is optional.

Options may use either - or / prefixes.

**Available command-line options:**

| | |
|---|---|
| -connection <*type*> | Connection type (e.g., serialrtu, modbustcp, rtutcp) |
| -port <*name*> | Serial port name (e.g., COM1, COM6, …) |
| -baudrate ### | Serial port baud rate (e.g., 19200) |
| -parity <*mode*> | Serial port parity (e.g., odd, even, none, mark, space) |
| -endofmessagetimeout ### | End-of-message timeout for serial communication (ms) (e.g., 100) |
| -onewire | Enables 1-wire operation (ignores query/command echo) |
| -ipaddress <*address*> | Slave/server IP address (e.g., 192.168.1.100) |
| -ipport ### | Slave/server TCP port (e.g., 502) |
| -slaveid ### | Slave ID (1–255) |
| -maxframe ### | Maximum frame size (13–256 bytes) |
| -responsetimeout ### | Maximum wait time for the first response byte (ms) |
| -interframedelay ### | Delay before sending the next query (ms). 0 is 'auto'. |
| -writeregfc <*mode*> | Function code for writing Holding Registers: 6, 16, or auto |
| -samplerate ### | Polling period for reading registers (ms) |
| -? or -h or -help | Shows the command-line help screen |

The command-line parameters are applied **only at program startup**. They are **not** re-applied when the .mbc project file is reloaded or when a different project file is opened.

## 10 ModbusExport tool

**ModbusExport.exe** is a command-line tool used to export registers from a Modbus device to a **CSV file**. It exports all **readable registers** from all tabs of an `.mbc` project file — along with their current values — into a single CSV output.

See The Import/Export format for details on the CSV structure.

**Example usage:**

- *ModbusExport Test1.mbc Data1.csv*
- *ModbusExport Test1.mbc Data1.csv -connection modbustcp -ipaddress 192.168.1.100 -format int -log log1.txt*
- *ModbusExport Test1.mbc Data1.csv -tabs 'Base setup" "Setup" "Parameters"*

The **.mbc filename** (or full path) is **required**.

The **CSV filename** (or full path) is **required**.

Connection parameters defined in the `.mbc` file can be **overridden** using command-line options.

Options may use either – or / prefixes.

**Command-line options:**

| | |
|---|---|
| -connection <*type*> | Connection type: serialrtu, modbustcp, or rtutcp.<br>Default: serialrtu |
| -port <*name*> | Serial port name (e.g., COM1)<br>Default: COM1 |
| -baudrate ### | Serial port baud rate<br>Default: 9600 |
| -parity <*mode*> | Serial parity: odd, even, none, mark, space.<br>Default: None |
| -endofmessagetimeout ### | Serial port End-of-message timeout (ms)<br>Default: 100ms |
| -onewire | Enables 1-wire mode (ignores echo)<br>Default: off |
| -ipaddress <*address*> | Slave/server IP address (e.g., 192.168.1.201) |
| -ipport ### | Slave/server TCP port<br>Default: 502 |
| -slaveid ### | Slave ID (1–255)<br>Default: 1 |
| -maxframe ### | Maximum frame size (13–256 bytes)<br>Default: 256 |
| -responsetimeout ### | Maximum wait time for first response byte (ms)<br>Default: 1000 |
| -interframedelay ### | Delay before sending next query (ms)<br>Default: 0 (auto) |
| -retries ### | Retries after communication errors<br>Default: 0 |
| -int32endian <*endian*> | 32-bit integer word order: little or big<br>Default: Little |
| -float32endian <*endian*> | 32-bit float word order: little or big<br>Default: Little |
| -stringendian <*endian*> | String byte order: little or big<br>Default: Big |
| -format <*format*> | Export format: default, int, uint, or hex<br>Default: 'default': format as given in mbc file. |
| -stringformat <*format*> | String export format: string, int, uint, or hex<br>Default: string |
| -tabs <*tabname*> [*tabname*]... | Names of tab pages to export (case-insensitive; quotes allowed) |
| -log <*file/path*> | Append output to a log file.<br>Default: no logging |

| | |
|---|---|
| -logtime | Add timestamps to log entries |
| -cont | Continue on error when reading a register<br>Default: off |
| -verbose | Display verbose output.<br>Default: off |
| -? or -h or -help | Show the help screen |

**Exit codes:**

0 = No error
1 = Command-line error
2 = File error
3 = Communication error

## 11 ModbusImport tool

**ModbusImport.exe** is a command-line tool used to **import register values from a CSV file into a Modbus device**. It writes all **writeable registers** found in the CSV file to the device.

See The Import/Export format for details on the CSV file structure.

**Example usage:**

- *ModbusImport Test1.mbc Data1.csv*
- *ModbusImport Data1.csv -connection modbustcp -ipaddress 192.168.1.100*
- *ModbusImport Data1.csv -connection modbustcp -ipaddress 192.168.1.100 -tabs Setup*

The **.mbc filename** is **optional**.

- If provided, connection parameters (connection type, IP address, serial port, baud rate, etc.) are taken from the .mbc file.
- Connection parameters defined in the `.mbc` file can be **overridden** using command-line options.
- If **no** .mbc file is provided, **all necessary connection parameters must be specified** on the command line.

The **CSV filename** (or full path) is **required**.

Options may use either - or / prefixes.

**Command-line options:**

| | |
|---|---|
| -connection <*type*> | Connection type: serialrtu, modbustcp, or rtutcp<br>Default: serialrtu |
| -port <*name*> | Serial port name (e.g., COM1)<br>Default: COM1 |
| -baudrate ### | Serial port baud rate<br>Default: 9600 |
| -parity <*mode*> | Serial parity: odd, even, none, mark, space.<br>Default: None |

| | |
|---|---|
| -endofmessagetimeout ### | Serial port End-of-message timeout (ms)<br>Default: 100ms |
| -onewire | Enables 1-wire mode (ignores echo)<br>Default: off |
| -ipaddress <*address*> | Slave/server IP address (e.g., 192.168.1.201) |
| -ipport ### | Slave/server TCP port<br>Default: 502 |
| -slaveid ### | Slave ID (1–255)<br>Default: 1 |
| -maxframe ### | Maximum frame size (13–256 bytes)<br>Default: 256 |
| -responsetimeout ### | Maximum wait time for first response byte (ms)<br>Default: 1000 |
| -writeregfc <*mode*> | Function code for writing registers: 6, 16, or auto.<br>Default: auto |
| -interframedelay ### | Delay before sending next query (ms)<br>Default: 0 (auto) |
| -retries ### | Retries after communication error.<br>Default: 0 |
| -int32endian ### | 32-bit integer word order: little or big<br>Default: Little |
| -float32endian ### | 32-bit float word order: little or big<br>Default: Little |
| -stringendian ### | String byte order: little or big<br>Default: Big |
| -tabs <*tabname*> [*tabname*]... | Names of tab pages to be imported, in this order, case insensitive, tab names can be quoted.<br>Default: import all tab pages. |
| -log <*file/path*> | Append output to a log file.<br>Default: no logging |
| -logtime | Add timestamps to log entries |
| -cont | Continue on error when writing a register<br>Default: off |
| -verbose | Display verbose output.<br>Default: off |
| -? or -h or -help | Show the help screen |

**Exit codes:**

0 = No error
1 = Command-line error
2 = File error
3 = Communication error

## 12 The Import/Export format

ModbusControl can **export** register values to a CSV file and **import** register values from a CSV file.

Exported CSV files are written using **UTF-8 encoding**, and imported CSV files are expected to use UTF-8 as well.

Fields in the CSV file are separated by **semicolons**. This semicolon-delimited format is common in Western Europe and works well with typical Windows regional settings and Microsoft Excel.

Lines beginning with # are treated as **comments** and ignored during import.

Fields that contain a separator, carriage return, or linefeed must be enclosed in **double quotes (")**. If a field itself contains a double quote, the quote must be escaped by doubling it. For example: `"My name is ""John"""`.

This quoting and escaping format is fully compatible with Microsoft Excel.

Example import/export line:

```
40108;1;"Analog in V1 (0-1000)";r;UInt;861
```

Register number, usually 40001-49999 for modbus holding registers
Value size, 1, 2 or more registers
Register description text
"r" and/or "w" for read/write
Format (Int, UInt, Hex, Binary, Float, ASCII, String)
Value (can be signed, unsigned, hex, float, ASCII, string)

Import files can have 2, 3, 4 ,5 or 6 fields. They are used as this:

| | |
|---|---|
| 2 fields | Register, Value |
| 3 fields | Register, Size (1,2,…), Value |
| 4 fields | Register, Size, Description, Value |
| 5 fields | Register, Size, Description, RW, Value |
| 6 fields | Register, Size, Description, RW, Format, Value |

**Notes on older formats:**

Older import/export files specified the value width in bits (16 or 32). The format has now been updated to specify the number of registers instead, providing better support for string values.

## 13 More about Modbus

For more detailed information about the Modbus protocol, visit **www.modbus.org**. Useful resources include:

- **Technical Resources**
- **Modbus Specifications**
- **MODBUS Protocol Specification**
- **Modbus Serial Line Protocol and Implementation Guide**Editing the mbc project file

## 13.1 Editing the XML in a .mbc project file

The `.mbc` project file is an **XML-formatted text file**, which makes it both human-readable and machine-readable. ModbusControl does not (yet) support creating new project files, so they must be edited manually. You can also use the example project **Demo.mbc** — available from the **Help** menu — as a starting point.

Many text editors, such as **Notepad++**, automatically recognize XML files and apply syntax highlighting, which makes editing easier.

**XML tags are case-sensitive.** Current versions of ModbusControl use **all-lowercase XML tags**, while older versions used a mix of lowercase and camel-case tags such as “SampleRate”. Using all-lowercase tags is now recommended. For compatibility, ModbusControl still recognizes the older camel-case tags.

## 13.2 The <modbuscontrol> root element

Every `.mbc` project file must contain a single `<modbuscontrol>` **root element**. All other configuration elements are placed inside this root element.

The `<modbuscontrol>` element supports the following child elements:

| | |
|---|---|
| `<size>` | Defines the size of the ModbusControl window. |
| `<modbus>` | Configures communication parameters for the modbus device. |
| `<modbustcpserver>` | Configures the built-in Modus/TCP bridge. |
| `<autostart>` | Configures the auto-start behavior. |
| `<tab>` | Configures one or more tab pages containing modbus registers. |
| `<include>` | Allows one .mbc file to include the tab pages of another .mbc file. |

## 13.3 The <size> element

The `<size>` element defines the initial size of the ModbusControl window when the project is opened. This element is **optional**.

It may contain the following child elements:

| | |
|---|---|
| `<width>` | The default width of the program window. |
| `<height>` | The default height of the program window. |

On a **96-DPI** system, these values represent the window size in **pixels**. On **high-DPI** systems, the values are automatically scaled so that the window appears the same physical size as on a 96-DPI display.

Example:

```
<size>
   <width>580</width>
   <height>600</height>
</size>
```

### 13.4 The <modbus> element

The <modbus> element defines the communication settings for connecting to the Modbus device. All elements are **optional**. For additional details, see Project Configuration.

The <modbus> element may contain the following child elements:

| | |
|---|---|
| <connection> | Defines the connection type and protocol.<br>• serial/rtu — Serial port using Modbus RTU protocol<br>• modbus/tcp — TCP connection using Modbus/TCP protocol<br>• rtu/tcp — TCP connection using Modbus RTU protocol |
| <port> | Serial port name (e.g., COM1). |
| <baudrate> | Serial speed in bits per second (e.g., 9600, 19200). |
| <parity> | Parity setting: None, Even, Odd, etc. |
| <endofmessagetimeout> | End-of-message timeout (ms). Required when using the Modbus/TCP bridge with unknown function codes. Default: 100 ms. |
| <onewireechosuppression> | Suppresses echo in 1-wire mode. Values: true or false. |
| <ipaddress> | IPv4 address (e.g., 192.168.1.52 or 127.0.0.1). |
| <ipport> | TCP port number. Modbus/TCP typically uses port **502**. |
| <slaveid> | Slave ID (1–255). Also called *slave address* or *unit identifier*. |
| <responsetimeout> | Maximum wait time for the first response byte (ms). |
| <maximumframesize> | Maximum allowed query or response frame size (13–256 bytes). Helps support devices with small serial buffers. |
| <interframedelay> | Delay between receiving a response and sending the next query. Modbus slaves on a serial multidrop network like RS485 require this delay, as defined in the modbus specs.<br>• Serial: 0 = default delay (4 character times). Values > 0 = delay in ms.<br>• TCP: 0 = no delay. Values > 0 = delay in ms.<br>Can also be used to reduce load on the slave/server. |
| <writeholdingregisterfc> | Defines which function code is used for writing Holding Registers:<br>• 6 or 06 — Write Single Register<br>• 16 — Write Multiple Registers<br>• auto — FC6 for 16-bit values, FC16 for multi-register values (recommended) |
| <int32endian> | Word order for 32-bit integers:<br>• little — low-high word order<br>• big — high-low word order<br>Replaces the older <Value32BitEndian> element. |

| | |
|---|---|
| `<float32endian>` | Word order for 32-bit floating-point values:<br>• little — low-high<br>• big — high-low |
| `<stringendian>` | Byte order for strings stored in registers:<br>• little — low byte first<br>• big — high byte first |
| `<stringencoding>` | Encoding used for strings. Common values: us-ascii, UTF-8, Windows-1252. |
| `<holdingregisterstart>` | Defines the numbering scheme used for Holding Registers.<br><br>40001:<br>The most commonly used scheme. Ranges **40001–49999** and **400001–465536** are supported and can be used simultaneously. Both **40001** and **400001** refer to the first holding register, which is addressed as **0000** in the protocol.<br><br>40000:<br>Ranges **40000–49999** and **400000–465535** are supported and can be used simultaneously. Both **40000** and **400000** refer to the first holding register, addressed as **0000** in the protocol.<br><br>400001:<br>Six-digit numbering scheme. Range **400001–465536** is supported. **400001** refers to the first holding register, addressed as **0000** in the protocol.<br><br>400000:<br>Six-digit numbering scheme. Range **400000–465535** is supported. **400000** refers to the first holding register, addressed as **0000** in the protocol. |

Example:

```
<modbus>
  <connection>serial/rtu</connection>
  <port>COM1</port>
  <baudrate>19200</baudrate>
  <parity>none</parity>
  <endofmessagetimeout>100</endofmessagetimeout>
  <slaveid>1</slaveid>
  <responsetimeout>1000</responsetimeout>
  <maximumframesize>256</maximumframesize>
  <writeholdingregisterfc>16</writeholdingregisterfc>
  <int32endian>little</int32endian>
  <float32endian>little</float32endian>
  <ascii16endian>little</ascii16endian>
  <samplerate>1000</samplerate>
</modbus>
```

## 13.5 The <tab> element

Each `<tab>` element defines a **tab page** in the ModbusControl user interface. A tab page has a **name** and contains a list of **variables**, where each variable represents a Modbus register with its description, size, numeric type, and display format.

A <tab> element may contain the following child elements:

| | |
|---|---|
| <name> | The name of the tab page, for example "Temperatures". |
| <exportorder> | Defines the order in which tab pages are exported to CSV. This is useful when you want registers to appear in a specific sequence — for example, exporting setup registers first to ensure correct import behavior.<br>• Must be a **positive integer** (e.g., 0, 1, 2, 100, 200, … up to 4294967295).<br>• Tabs are exported in **ascending order** of this number.<br>• Tabs **without** an <exportorder> value are exported **after** all numbered tabs, in the order they appear in the .mbc file.<br>• Optional. |
| <fcol> | Sets the **default font (text) color** for all <var> elements on this tab page. See The <fcol> and <bcol> elements for details. Optional. |
| <bcol> | Sets the **default background color** for all <var> elements on this tab page See The <fcol> and <bcol> elements for details. Optional. |
| <expandfields> | Controls whether bit fields of all <var> elements on this tab page are automatically expanded when the project is loaded.<br>• Valid values: true or false<br>• Default: false<br>• Optional |
| <var> | Defines one or more Modbus registers displayed on this tab page. Each <var> includes: register number, size (number of registers), description text, numeric type and format, colors, and more. |

Example of a <tab> element:

```
<tab>
<name>Inputs (read only)</name>
<var><reg>40001</reg><text>Software version Major</text><type>int16</type>
  <rw>r</rw></var>
<var><reg>40002</reg><text>Software version Minor</text><type>int16</type>
  <rw>r</rw></var>
</tab>
```

## 13.6 The <include> element

The <include> element allows one .mbc file to **include the tab pages and registers** defined in another .mbc file. Only the **tab-related elements** of the included file are imported — other elements such as <size> and <modbus> are **ignored**.

This mechanism is useful for maintaining shared sets of tab pages (for example, common setup registers) in a separate project file. An .mbc file may contain **multiple** <include> elements, and each included file may itself contain additional <include> elements.

The <include> element may contain the following child elements:

| | |
|---|---|
| <file> | Specifies the filename or full path of the .mbc file to include. |
| <required> | Controls whether an error is reported when the referenced file cannot be found.<br>• true — An error is reported if the file is missing.<br>• false — No error is reported if the file is missing.<br>• Default: **true** |

Examples of <include> elements:

```
<include>
   <file>Configuration.mbc</file>
   <required>false</required>
</include>

<includex>
   <file>C:\Projects\Special\Calibration.mbc</file>
</include>
```

### 13.7 The <var> element

Each <var> element defines **one variable** (one or more Modbus register) that appears as a **single variable (row)** in the ModbusControl grid.

A <var> element may contain the following child elements:

| | |
|---|---|
| <reg> | The Modbus register number. The numbering scheme depends on the <holdingregisterstart> setting. Holding Registers typically start at **40001**. |
| <text> | A short description of the register. See The <text> element for details. |
| <longdescr> | A longer descriptive text shown as a **tooltip** when the user hovers over the description. Supports **multi-line** text using CR/LF. |
| <type> | Defines the register's data type, including:<br>• Size (16-bit or 32-bit)<br>• Signed/unsigned interpretation<br>• Display format (decimal, hex, etc.)<br>See The <type> element for details. |
| <minlen> | Specifies the **minimum number of digits** shown for the value. Leading zeros are added if needed.<br>Useful for binary or hexadecimal formats to prevent the value from shifting horizontally as bits change.<br>• Example: value 15 (decimal), hex format, minlen=4 → 0x000F<br>• Optional<br>• Default: 0 |
| <rw> | Defines whether the register is:<br>• r — read-only<br>• w — write-only<br>• rw — read/write<br>ModbusControl reads only registers marked with 'r'. Allows writing only to registers marked with 'w'.<br>Optional. Default: r |
| <fcol> | Sets the **font (text) color** for the register row. See The <fcol> and <bcol> elements for details. Optional. |
| <bcol> | Sets the **background color** for the register row. See The <fcol> and <bcol> elements for details. Optional. |
| <decimals> | Displays the value with a fixed number of decimal places.<br>• Allowed only for **Int** and **UInt** types |

- Valid range: **0–12**
- Optional
- Default: **0**

Example: Value 12345 with decimals=2 → 123,45

| | |
|---|---|
| `<enum>` | Enumeration defines a list of **named constants** for specific values. ModbusControl displays the matching enum text **after** the numeric value in the value column.<br>See The <enum> element for details. Optional. |
| `<field>` | Defines one or more **bit fields** within the register. When bit fields are present, the register can be **expanded** to show each bit field separately. Useful for registers that contain bit-mapped digital inputs or status flags.<br>See The <field> element for details. Optional. |
| `<expandfields>` | Controls whether this register's bit fields are automatically expanded when the project loads.<br>• Valid values: true, false<br>• Default: **false**<br>• Optional |

Example of a simple `<var>` element:

```
<var><reg>40001</reg><text>Counter 1</text><type>int16</type>
 <rw>rw</rw></var>
```

Example of a `<var>` element with one decimal and a light red background:

```
<var><reg>40025</reg><text>Temperature 1 (0,1 deg)</text><type>int16</type>
 <decimals>1</decimals><rw>r</rw><bcol>255,200,200</bcol>
 </var>
```

## 13.8 The <text> element

The `<text>` element contains the description text shown next to a register. Because XML reserves certain characters, you must use **escape sequences** when including characters that cannot appear directly in XML.

XML defines **five predefined entities**:

| | | |
|---|---|---|
| `<` | represents | < |
| `>` | represents | > |
| `&` | represents | & |
| `'` | represents | ' |
| `"` | represents | " |

All other special characters must be written using **numeric character references**. Any valid Unicode character can be represented this way.

```
<text>Temperature 1 (&#176;C)</text>
```
shows "Temperature 1 (°C)"

Extensive lists of XML/HTML character entities and Unicode code points can be found on the internet.

## 13.9 The <longdescr> element

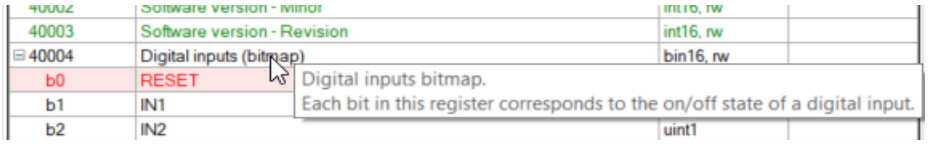
The `<longdescr>` element defines an extended description text that is displayed as a **tooltip**. Tooltips are commonly used to provide additional information when the user hovers the mouse pointer over an element.

`<longdescr>` Sets the tooltip text for the register.

- Supports **multi-line** text using standard CR/LF characters.
- A line break can also be written using an XML numeric escape sequence:  (This inserts a carriage return.)

For example:

```
<var>
  <reg>40004</reg>
  <text>Digital inputs (bitmap)</text>
  <longdescr>Digital inputs bitmap. &#13;Each bit in this register corresponds to the
on/off state of a digital input.</longdescr>
  <type>bin16</type>
```




## 13.10 The <type> element

The `<type>` element defines the **data type** of the register. This determines:

- Whether the integer-type value is **16-bit** or **32-bit** width. For a string variable the number of **Modbus registers** that together form the full value of the variable.
- How the value is **interpreted** (signed, unsigned, float, ASCII, string…)
- How the value is **displayed** (decimal, hexadecimal, binary…)

The `<type>` element defines the type of the register. This defines the size of the register as 16 or 32-bit, and the display format like signed, unsigned, hexadecimal, etc...

Supported types:

| | |
|---|---|
| `int16` | Signed 16-bit integer |
| `uint16` | Unsigned 16-bit integer |
| `int32` | Signed 32-bit integer (uses two 16-bit registers) |
| `uint32` | Unsigned 32-bit integer (uses two 16-bit registers) |
| `hex16` | 16-bit hexadecimal value (e.g., 1000 → 0x3E8) |
| `hex32` | 32-bit hexadecimal value (uses two registers) |
| `bin16` | 16-bit binary value (e.g., 1000 → 0b1111101000) |
| `bin32` | 32-bit binary value (uses two registers) |
| `float32` | 32-bit IEEE-754 floating-point value (uses two registers) |

| | |
|---|---|
| `ascii16` | Two ASCII characters stored in one register (e.g., 0x4142 → "AB") |
| `ascii32` | Four ASCII characters stored in two registers (e.g., 0x43444142 → "ABCD") |
| `string2/4/6 .. 300` | Text string, length 2–300 bytes (1–150 registers).<br>The byte order and the string encoding/decoding format are defined in the project configuration.<br>The string is **zero-terminated**, meaning it consists of a sequence of characters followed by a byte with value **0**, which marks the end of the string.<br>For example, the string **"AB"** is decoded from two registers with the values **0x4142** and **0x0000**.<br>A **string40** type provides space for **40 bytes**, which corresponds to **39 characters (assuming single-byte characters) plus a terminating zero**. |

## 13.11 The <enum> element

The `<enum>` element defines a list of **named values**, similar to a set of constants. When a register's value matches one of the defined items, ModbusControl displays the corresponding **enum text** after the numeric value in the value column.
An `<enum>` element contains one or more `<item>` elements. Each `<item>` defines:

- a **value**
- the **text** that should be displayed for that value.
-

For example, this enum defines texts for the values 0, 1, 2, 3 and 4.

```
<enum>
       <item><value>0</value><text>no alarm</text></item>
       <item><value>1</value><text>T1 FAIL</text></item>
       <item><value>2</value><text>T1 TOO HOT</text></item>
       <item><value>3</value><text>T2 FAIL</text></item>
       <item><value>4</value><text>T2 TOO HOT</text></item>
       <else>UNRECOGNIZED VALUE!</else>
</enum>
```

Each **value** may appear **only once** within an `<enum>`.

The `<else>` element is **optional**.

If the register value does **not** match any `<item>`:

- If an `<else>` element is present, its text is shown.
- If no `<else>` element is present, ModbusControl displays **"(enum ?)"**.

The `<else>` element may be left **empty** to suppress the message entirely.

## 13.12 The <field> element

Each `<field>` element defines a **bit field** within a register. A bit field may have a width of **one bit or multiple bits**. When a register contains one or more bit fields, it can be **expanded** in the interface to show each field's value separately — useful for inspecting digital inputs or status flags.

A `<field>` element may contain the following child elements:

| | |
|---|---|
| `<pos>` | Specifies the **position of the lowest bit** of the bit field. 0 = least significant bit of the register. |

| | |
|---|---|
| `<width>` | Defines the **number of bits** in the bit field. Optional. Default: **1.** |
| `<text>` | The description of the bit field. See The <text> element for details. |
| `<longdescr>` | A longer descriptive text shown as a **tooltip** when the user hovers over the bit field description.<br>• Supports multi-line text using CR/LF<br>• Line breaks may also be written using the XML escape  |
| `<type>` | Defines how the bit field value is displayed:<br>• int — signed decimal<br>• uint — unsigned decimal<br>• hex — hexadecimal<br>• bin — binary<br>Optional. Default: uint |
| `<minlen>` | Specifies the **minimum number of digits** shown for the value. Leading zeros are added if needed.<br>Optional. Default: **0.** |
| `<fcol>` | Sets the **font (text) color** for the bit field row. See The <fcol> and <bcol> elements for details. Optional. |
| `<bcol>` | Sets the **background color** for the bit field row. See The <fcol> and <bcol> elements for details. Optional. |

Example of a `<field>` element, defines 1 bit:

```
<field><pos>0</pos><text>RESET button</text></field>
```

Example of a `<field>` element with width 4, on bits 11..8, shown in hex:

```
<field><pos>8</pos><width>4</width><text>Mode</text><type>hex</type></field>
```

Example of a `<var>` element with bit fields:

```
<var><reg>40009</reg><text>Digital inputs (bitmap)</text>
 <type>bin16</type><minlen>8</minlen><rw>r</rw>
 <field><pos>0</pos><text>RESET button</text></field>
 <field><pos>1</pos><text>IN1</text></field>
 <field><pos>2</pos><text>IN2</text></field>
 <field><pos>3</pos><text>IN3</text></field>
</var>
```

### 13.13 The <fcol> and <bcol> elements

The `<fcol>` and `<bcol>` elements define the **font (text) color** and **background color** for registers, bit fields, or tab pages. A color can be specified in **two different formats**:

- **RGB values** (decimal R,G,B)
  Specify the **red**, **green**, and **blue** components as three integers in the range **0–255**, separated by commas.
  For example:

  | | |
  |---|---|
  | `<bcol>255,0,0</bcol>` | Bright red background |
  | `<bcol>0,255,0</bcol>` | Bright green background |
  | `<bcol>255,255,180</bcol>` | Soft yellow background |
  | `<bcol>200,200,200</bcol>` | Grey background |

- **HTML color notation (**#RRGGBB**)**

Specify the color using a hexadecimal HTML color code:
RR = red component (00–FF)
GG = green component (00–FF)
BB = blue component (00–FF)
For example:

| | |
|---|---|
| `<bcol>#FF0000</bcol>` | Bright red background |
| `<bcol>#00FF00</bcol>` | Bright green background |

## 13.14 The <autostart> element

The `<autostart>` element controls whether communication with the Modbus device is started **automatically** when the project is opened, and optionally which **tab page** is shown first.

If no `<autostart>` element is present:

- Communication **starts automatically**
- The **first tab page** is shown

When an `<autostart>` element is defined, the autostart behavior is determined by its child elements.

A `<autostart>` element may contain the following:

| | |
|---|---|
| `<enable>` | Enables or disables automatic start of communication.<br>Valid values: true, false<br>Optional. Default: **false** |
| `<page>` | Specifies which tab page is shown when autostart is active.<br>Can be the **name** of a tab page (e.g., "Inputs") or the **tab page number**, starting from **1.**<br>Optional. Default: **first tab page** |

## 13.15 The <modbustcpserver> element

The `<modbustcpserver>` element configures the built-in **Modbus/TCP server**, which acts as a **bridge** between ModbusControl and the connected Modbus device. This allows external Modbus/TCP clients to access the same serial Modbus device **simultaneously**, effectively turning ModbusControl into a Modbus/TCP-to-RTU gateway.

For more information, see Modbus/TCP bridge.

A `<modbustcpserver>` element may contain the following child elements:

| | |
|---|---|
| `<enable>` | Enables or disables the Modbus/TCP bridge.<br>Valid values: true, false.<br>Optional. Default: **false** |
| `<port>` | Specifies the TCP port on which the Modbus/TCP server listens.<br>Optional. Default: **502** (the standard Modbus/TCP port) |

## 13.16 XML comments

XML comments allow you to **temporarily disable** parts of the project file — such as tabs or registers — without deleting them. ModbusControl ignores any content placed inside an XML comment.

An XML comment:

- **starts with:** `<!--`
- **ends with:** `-->`

In the example below, two `<var>` lines are commented out and therefore ignored by ModbusControl:

```
<var><reg>40006</reg><text>Analog in 1</text><type>uint16</type><rw>r</rw></var>
   <!--
   <var><reg>40007</reg><text>Analog in 2</text><type>uint16</type><rw>r</rw></var>
   <var><reg>40008</reg><text>Analog in 3</text><type>uint16</type><rw>r</rw></var>
   -->
<var><reg>40009</reg><text>Analog in 4</text><type>uint16</type><rw>r</rw></var>
```